

It's not about what you know but who you know

Scaling Credeq's engineering operations with agents by distributing the knowledge and centralizing only the routing

CREDEQ
1st Floor - 5 St Helen's Place – London - EC3A 6AB - United Kingdom
credeq.com

Index

1. The Trade
2. Framing the Lifecycle
3. Problem Statement
4. An Agent Is Just an Application
5. The Framework
6. The Proposed Architecture

The Trade

The good engineers build tools. The great ones build tools that build leverage, systems that let one person's judgment do the work of ten pairs of hands. None of this is new. Documentation, linters, CI pipelines, and now agents are the same instinct wearing a new outfit.

Credeq's data platform runs on multiple repositories. Each one is its own little kingdom, run by engineers who know its scar tissue better than anyone else. Now put AI agents to work inside those repos. They show up smart. What they don't show up with is the local knowlege, and the particular scar tissue of this repo, this region, the one config nobody ever bothered to write down because everyone just knew. That gap is real, and I'll save you the suspense, it's not the interesting part.

The interesting part is that the constraint mentioned above never left. Knowing which tool to use was always the engineer's job. Not the tool's. The judgment doesn't leave when the agent arrives. Someone still decides whether this is even a job for one, which agent, with what context, on which repo. Skip that decision and you haven't gotten rid of the skill requirement. The interesting question was never whether agents need context, of course they do, everything does. It's *where that context should live* and, the part everyone would rather skip, who's still on the hook for knowing when the right move is not to reach for an agent at all. That judgment call is what this document tries to bottle. Not more enthusiasm about agents but a framework, a way for any engineer to decide where an agent belongs and where it does not.

The second constraint is the one that hits the budge. It is a old well known tradoff in engineering.

Speed, **Quality** and **Cost**, only two are ever yours at once.

Scale doesn't repeal that law, it just closes the loopholes. So engineers are forced to choose between shipping fast and keeping things healthy, and whichever they pick, the other one quietly waits its turn. So here's the actual challenge, and it isn't "add more agents." It's how you get faster without quietly paying for it in quality, or in cash, later.

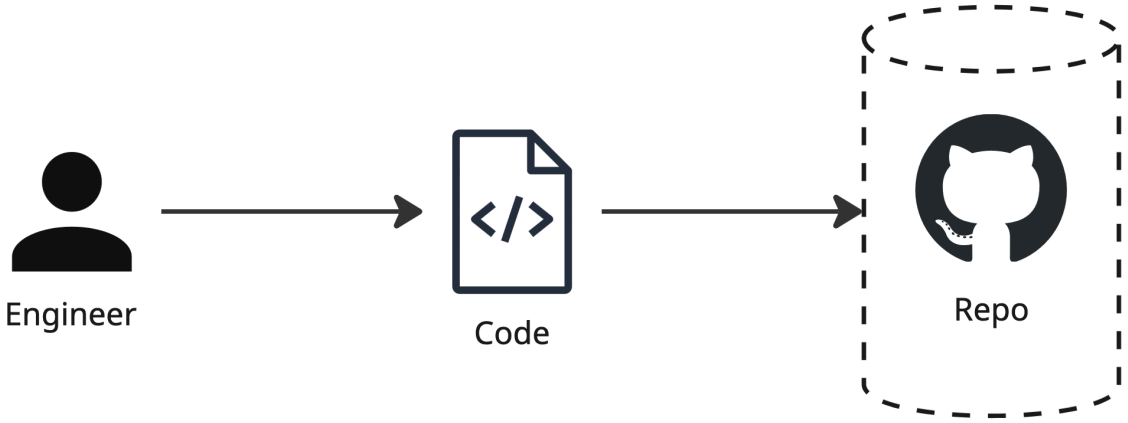
More agents, aimed right, is one way through that trade. Not one agent for several repos carrying every rule Credeq has ever written. Agents built for the repo they live in, carrying only what that repo needs, and nothing else. That's the shape of the proposed fix. What's still missing is the deciding part, which workflows actually deserve an agent and which don't. That starts with finding where engineers lose their time in the first place.

Framing the Lifecycle

To truly understand how we can scale engineering operations, we first need to agree on what an engineer actually does, and what an engineering operation is.

Strip away the job titles, the standups, the GitHub issues, and it comes down to, a engineer that writes code, and gets that code into a repository. Everything else, the skills they've picked up, the tools they reach for, exists to serve that one act. A linter, a framework, a snippet library, an agent, all of it is just

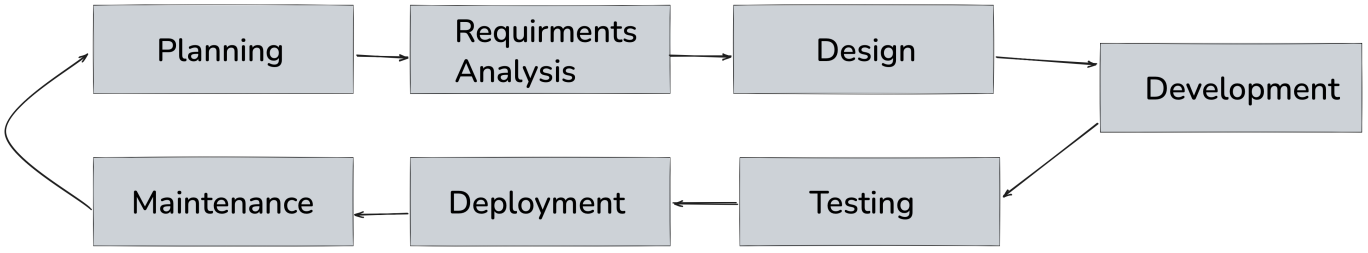
scaffolding around the same two steps. Write the code. Push the code.



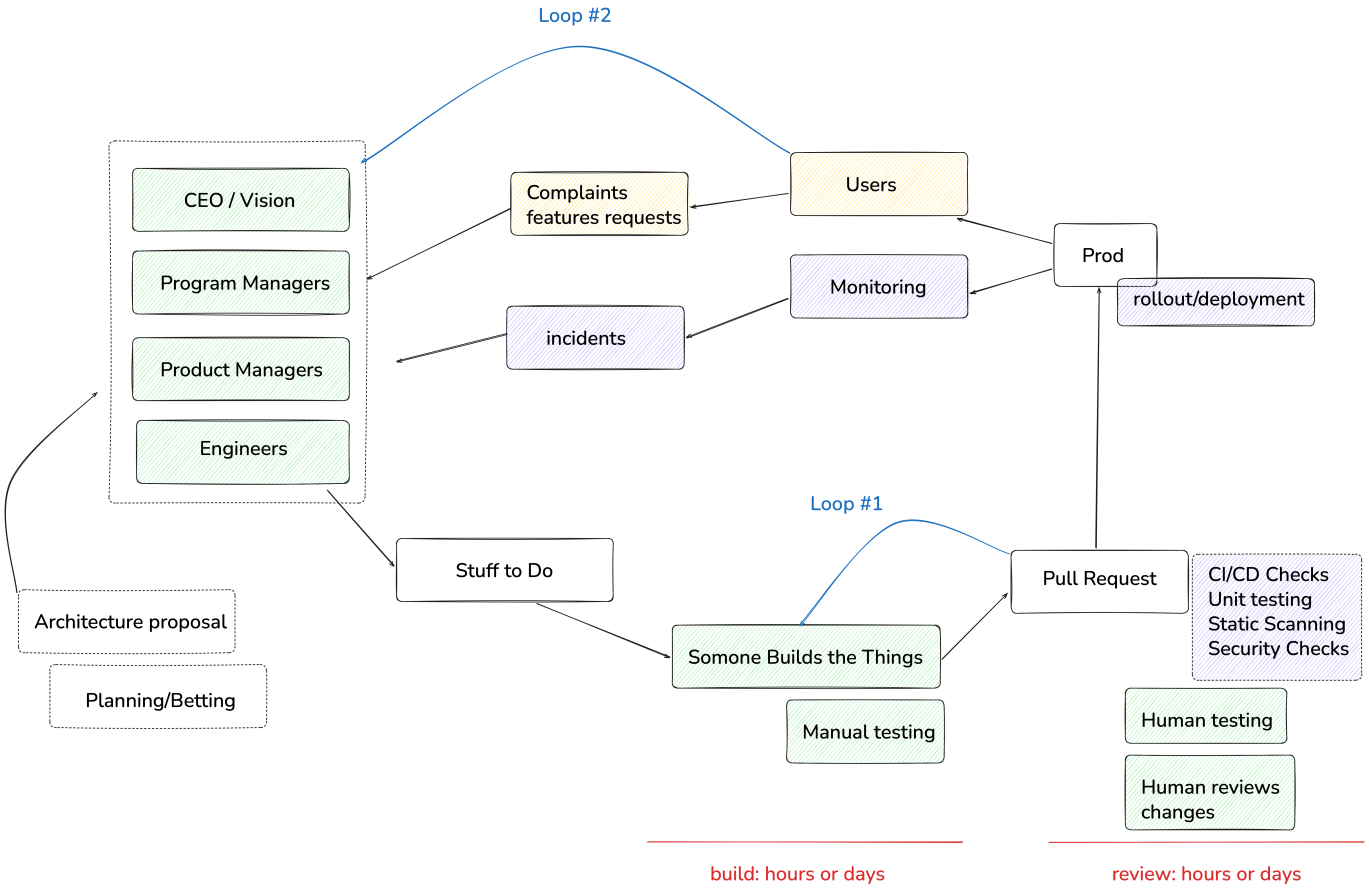
An oversimplification, sure. But a first principle is not the whole picture, it's the part that's true no matter what you strip away. Strip engineering all the way down and it's two steps, solve problems by writing code and share that code. Everything else is a decoration.

In practice, that decoration is most of the job. The loop grows steps around the core: plan the change, write it, test it, review it, merge it, ship it, watch it in production, then start over the moment something breaks or something new needs building. That longer loop has a name. It's called the software development lifecycle, and it's where the real bottlenecks live.

The Software Development Lifecycle

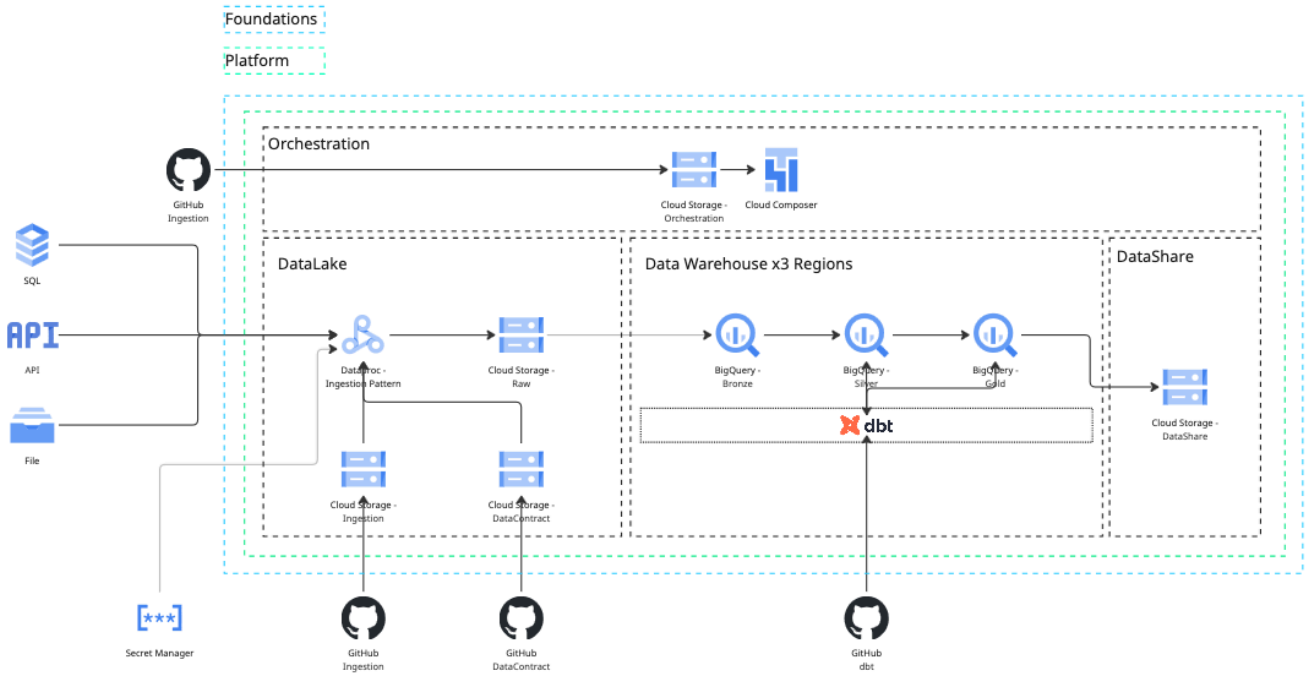


The lifecycle itself is not the problem, the loop is fine, it's been fine for decades. The problem is that each of those seven boxes is actually a bundle of a dozen smaller workflows wearing one label, writing tests versus running them, documentation versus code, a hotfix versus a rewrite. Seven boxes was always a map, but a engineer fills the gaps with judgment without noticing agents can't, not until someone breaks each box down into the workflows actually living inside it.



This lifecycle is not even the entire picture, this just shows you that those original boxes, in a production setting, turn into a whole intricate and complicated set of workflows.

Scaling the SDL to a Global Data Platform



Credeq's global data platform is multiple repositories, and each one runs its own version of that lifecycle, at its own pace, bottlenecked in its own place, for its own reasons.

- **dbt Region** — dbt modules and transformation logic for a specified region
- **Data Platform Foundations** — the infrastructure as code underneath all of it
- **Data Platform** — the modules that build up the data warehouse itself
- **Data Orchestration** — the Airflow DAGs that keep everything running on schedule
- **Data Contracts** — the contracts that govern what's allowed in from ingestion sources

A slow step in dbt Africa isn't the same bottleneck as a slow step in Data Contracts. Before anyone reaches for an agent: define each of these lifecycles on its own terms, and break each one down to the workflow level.

Each lifecycle decomposes into a set of workflows, and that decomposition is what actually shows engineers where the bottlenecks are, and where tooling would strategically help remove them. That's the real unit of granularity, not the lifecycle, and not the repo, the workflow underneath both of them.

Do the decomposition honestly and the workflows sort themselves into two kinds. Most of them stay home. A new model in dbt Africa gets planned, written, tested, and shipped without ever leaving that repo. **Local workflows**, local bottleneck, local fix.

Some workflows don't respect the walls of the house. A new field from an ingestion source starts as a change in Data Contracts, gets picked up by Data Orchestration, lands in the warehouse through Data Platform, and only becomes real once a dbt region transforms it. One workflow, four repos, four owners, and nobody who can see the whole thing from where they're standing. That's a **cross-repo workflow**, and it fails differently than a local one. Not inside a repo, in the handoffs between repos.

Multiple lifecycles, each hiding multiple workflows, some local to a single repo, some stretched across all of them at once. One shared skill file, or one shared agent, can't see this picture, averaging that many different bottlenecks into a single view is how you end up fixing none of them. Hold onto the local-versus-cross-repo distinction, it comes back when the framework has to survive contact with both.

Problem Statement

Putting the problem statement into words.

Scale Credeq's developer operations with AI agents, without compromising on quality, while increasing engineering velocity.

Three things in one sentence, and each one can quietly kill the other two. Scale carelessly and quality erodes. Chase velocity and the repos closest to launch cut corners the others won't. Protect quality by slowing everything down and you've just renamed the problem, not solved it. This is the same trade from earlier, speed, quality and cost.

An AI agent is just another tool, not a strategy, but also not quite tha same category as a linter or a CI pipeline. Yet the tool is only as good as the **bottleneck** it gets pointed at. Aim it at nothing in particular and it's just an expensive way to feel busy.

"Real tools get treated like tools". Nobody installs a linter without knowing exactly what it's not allowed to break. Nobody keeps a CI pipeline that can't show you what it caught. A tool earns its place **twice**, once before it goes in, checked against the constraints, and once after, against a number somebody actually reads. Agents don't get an exemption because they can talk or use stats to "reason".

Problem Statement Components

What the problem statement is actually made of, the moving parts that everything above was building toward.

- **Engineers** — the people whose judgment decides which tool gets used, where, and whether an agent belongs in the loop at all.
- **Tool usage** — agents, CI, cli tools... anything that turns judgment into leverage.
- **Processes** — the workflows, global and local, that all of this actually has to run inside.

Problem Statement Constraints

What the solution isn't allowed to sacrifice to get there.

- **Quality** cannot be compromised.
- **Speed** should increase.
- **Cost** should be minimized.

These aren't three rules of the same kind. Quality and cost are gates, you check them going in, and a violation vetoes the agent. Speed can function as a scoreboard you check it coming out, to see if the whole implementation paid off. Gates before, scoreboard after, mix them up and you get agents that are very fast at making things worse.

An Agent Is Just an Application

Agent explanation

Google's architecture guidance defines an agent as an application that achieves a goal by processing input, performing reasoning with available tools, and taking actions. Read it twice, because the word doing the work is *application*. Not a colleague, not a strategy, an application with a model sitting where the business logic used to sit. The reasoning is new. The rest is the same anatomy every tool has, inputs it accepts, tools it's allowed to touch, and a goal somebody defined.

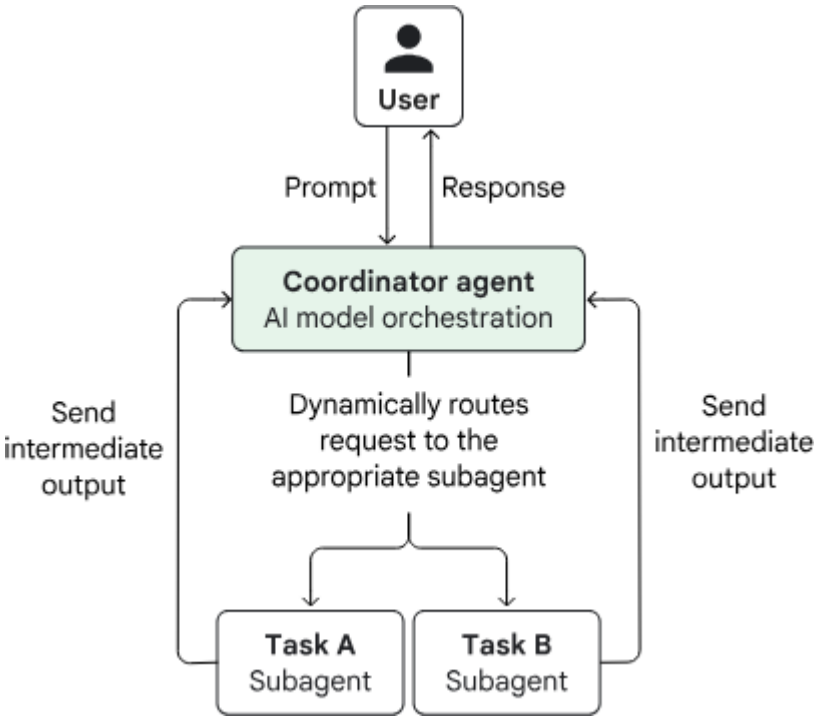


Design patterns to note

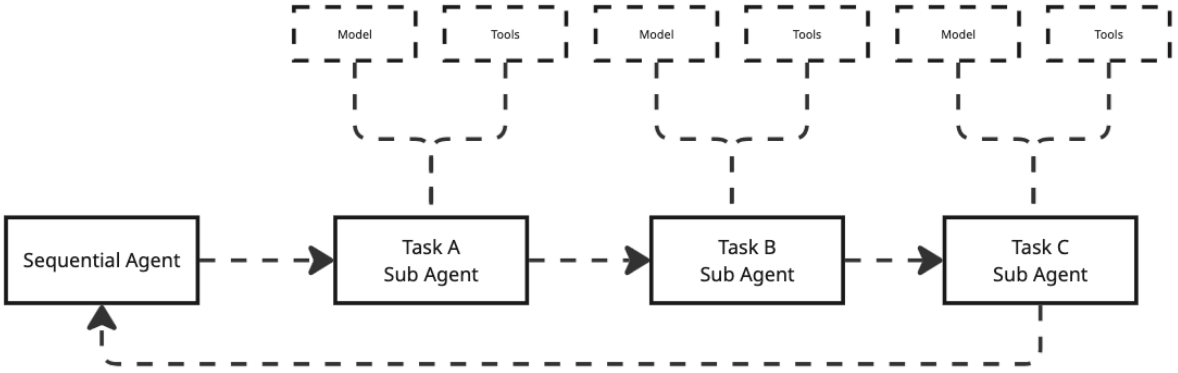
Orchestrator pattern

Google calls this the coordinator pattern, one central agent takes the request, breaks it into sub-tasks, and dispatches each piece to a specialized agent. The coordinator doesn't do the work. It routes the

work. That's useful precisely when the routing itself needs judgment, which is what a cross-repo workflow looks like from the outside.



Its stricter cousin is the **multi-agent sequential pattern**. This pattern includes specialized agents in a fixed order, each one's output becoming the next one's input. Google's guidance recommends it for highly structured, repeatable processes, and the example they reach for is data pipelines.



Hyper-specific single agent

This is Google's single-agent system with the scope turned all the way down: one model, a defined set of tools, a system prompt that covers one workflow and nothing else. Small enough to reason about, cheap enough to run, and it sidesteps the failure mode Google names tool bloat, tool definitions piling up in a system prompt until the model gets less accurate and more expensive at the same time. In a shared agent, every rule added for one repo is noise every other workflow has to pay for. Decomposing isn't a style preference. It's how the quality gate stays closed.

Two things follow, and they're easy to confuse. The **framework** is the *decision* — whether an agent belongs and where — and it can answer no. The **architecture** is the *build* — what you actually stand up,

but only for the workflows the framework didn't already reject. Framework first, because there's no point designing a system for an agent that shouldn't exist.

The Framework

Everything above collapses into three steps. Run them in order, every time, for every agent, and the judgment becomes a procedure anyone can run, not a call only the seniors can make

1. **Identify the SDL.** Which lifecycle are you standing in, and is it local to one repo or global to the platform.
2. **Identify the workflow.** Not the box on the diagram, the actual workflow living inside it.
3. **Check the gates.** Check the gates. Verify an agent at that exact point violates neither the cost nor the quality gate, speed comes later, on the scoreboard. If it violates one, stop. **No is a fine answer.**

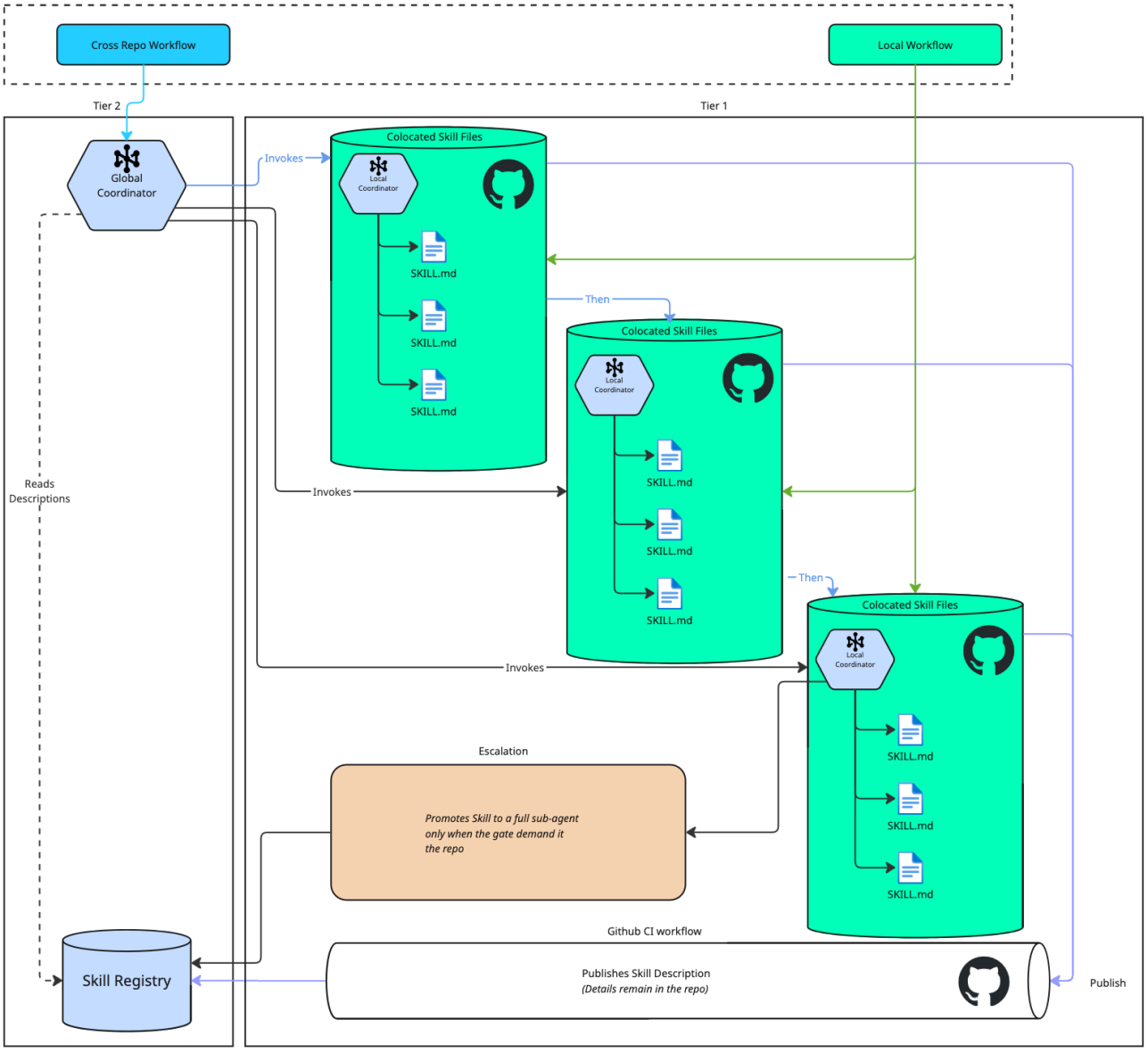
These aren't house rules. Google's own guidance on choosing agent design patterns starts from the same axes, the shape of the task, the cost budget, how much human oversight the job demands, before any pattern gets picked.

The Proposed Architecture

The architecture is two tiers, and they map directly onto the local-versus-cross-repo distinction from earlier, **distribute the knowledge, centralize only the routing.** By default the specialists aren't standalone agents at all, they're **skill files colocated in the repos they serve**, each carrying only its own repo's scar tissue(logic) and versioned alongside the code it acts on.Execution happens in a coordinator, which does one thing, it routes.

Locally, a repo-local coordinator routes among a repo's own skills, however across repos, a central coordinator invokes each repo's skills in handoff order. Either way the coordinator knows nothing about the domain. It routes; the skills know how. Never the reverse.

The coordinator succeeds not by knowing things, but by knowing who does.



A **local workflow** (green) is handled entirely inside its repo, its own coordinator invoking its own skills, no central hop, which keeps the common case cheap. A **cross-repo workflow** (blue) hits the central coordinator, which reads the skill registry and invokes the colocated skills that own the domain knowledge, in handoff order.

The registry doesn't fill itself, and it isn't filled by hand. The purple path is how a repo surfaces a skill, on merge, a CI step publishes *only that skill's description* to the central registry, never its body. The knowledge stays in the repo while the "pointer" travels. Two things fall out of that. - The registry is current the instant a PR lands. - Governance is free, it rides on the same CODEOWNERS review that already gates the code.

With this the thesis holds literally, **distribute the knowledge, centralize only the routing metadata**. The orange is a *different* axis and shouldn't be confused with registration. Escalation changes what a skill **runs as**, a full sub-agent, only when a gate demands something a file can't give. Registration changes how a skill is **found**. One is about runtime, the other about discovery.

Do the Specialists Need to Be Agents? Skills as the Default

A skill isn't an agent. An agent is an application with its own model, tools, and reasoning loop. A skill is passive knowledge, a file that gets loaded into a running agent's context when it's invoked, and does nothing on its own. So Tier 1 isn't really "agents living in repos." It's knowledge files living in repos, plus one host agent that loads the right one at the right time. The knowledge stays distributed; the execution centralizes into the host. That's the thesis.

Framed as skills, most of the argument gets *stronger*. **Ownership and colocation** stop being aspirations.A skill is just a markdown file in the repo, versioned with the code, reviewed in the same PR, owned by the same CODEOWNERS, with no separate service to deploy or secure. **The registry comes for free**, it's just the set of skill descriptions the host can see. Discoverability isn't something we have to build, it's something we already have. And **the cost gate holds through progressive disclosure**. Progresive Disclosure allows the one-line description to stay in context cheaply, the body loads only on invocation, so only the relevant repo's rules ever enter the prompt. That last one has a condition attached, and it matters, the win holds only if skills are *invoked*, not preloaded. Concatenate every repo's skill into one always-on prompt and you've quietly rebuilt the shared mega-agent this whole document argues against.

Two guarantees get *weaker*, and the honest move is to name them rather than let them leak. **Isolation** is the first. Separate agents each run with their own tool permissions, a hard sandbox. Skills all load into the same host's context and share its tool access, so "a narrow specialist can't confidently wander into another repo" stops being an architectural wall and becomes a matter of selective invocation plus a harness that enforces per-skill tool allowlists. If the runtime doesn't scope tools per skill, that guarantee is just a prompt asking nicely. **Independent per-stage execution** is the second. With full agents the handoff is agent-to-agent, and each stage can carry its own model, permissions, and failure domain. With skills the handoff is one host swapping files in sequence, which is fine for the strictly sequential pipeline Credeq actually runs, but it buys no parallelism and no independent failure between stages.

So the rule is, **default to skills, promote to a full sub-agent only where a gate demands it**, its own tools, real parallelism, or independent failure isolation. That's the same "no is a fine answer" discipline from the framework, pointed at the runtime instead of the workflow: don't stand up an agent where a file does the job.

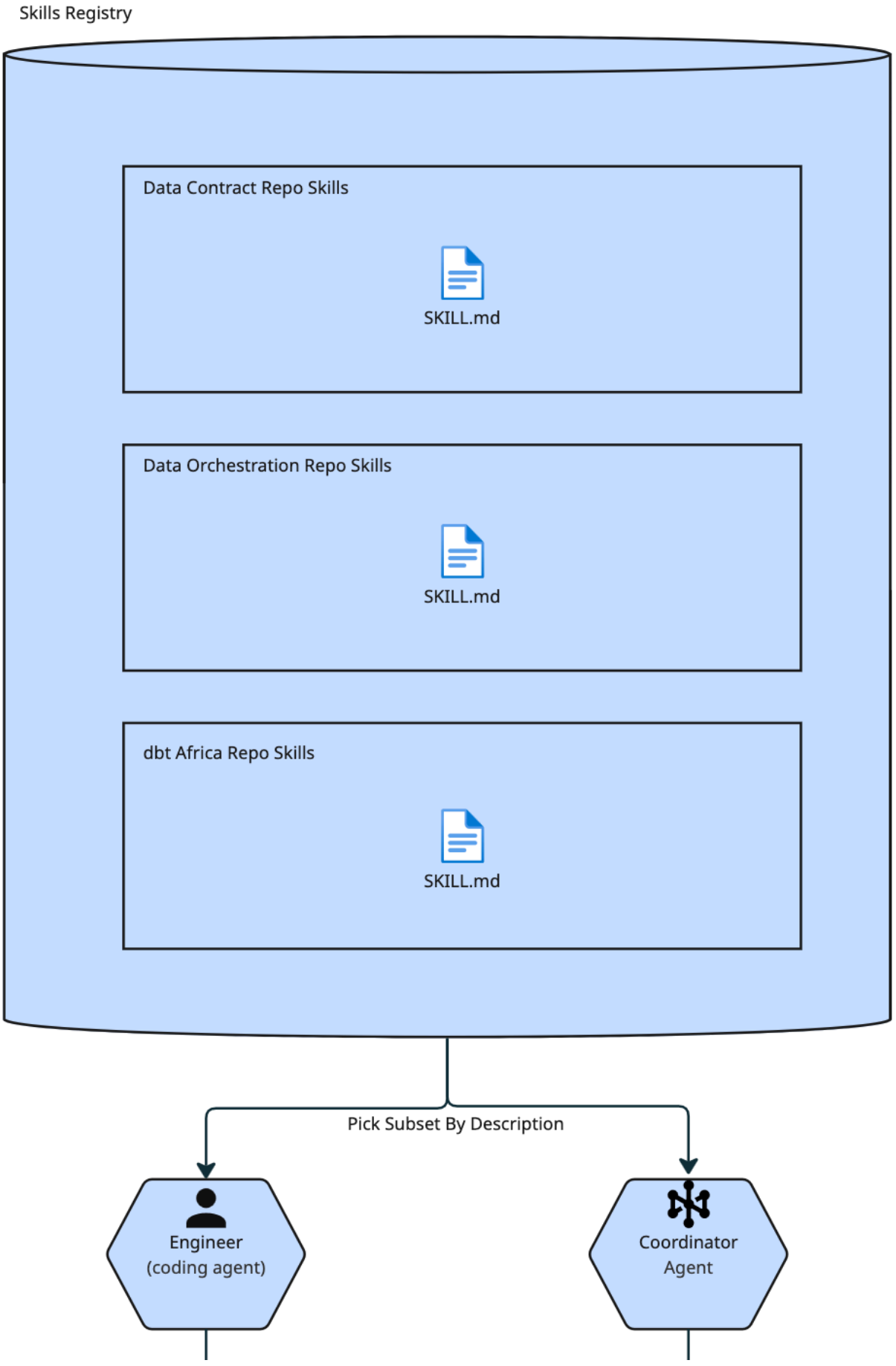
Composing New Agents From the Registry

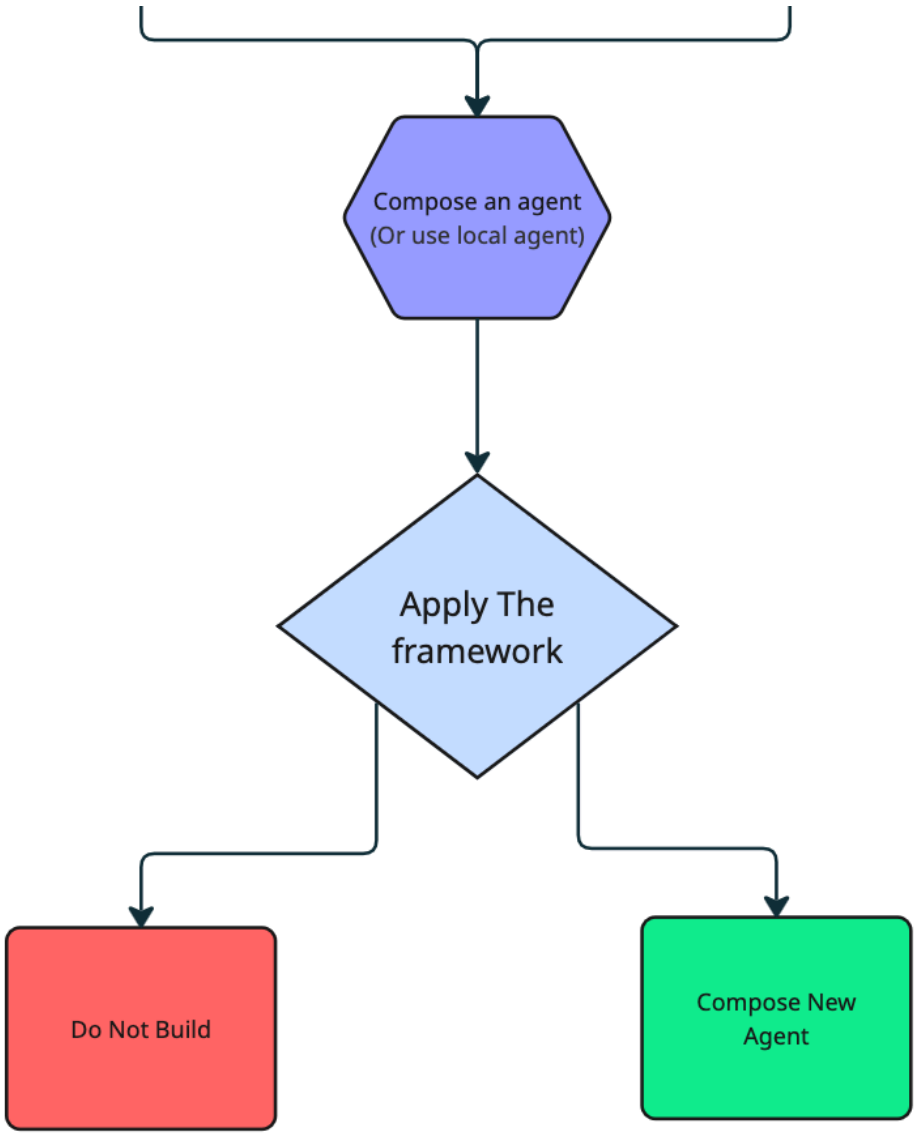
Once every skill is discoverable by description, the registry stops being just a routing table and becomes a **menu you compose from**. Defining a new agent no longer means authoring new knowledge. It means three cheap things: - Pick a subset of skills from the registry - Write a system prompt that states the goal - Declare a tool allowlist.

The knowledge already exists, owned and versioned in the repos that wrote it. The new agent is a *selection over the registry*, not a new body of rules.

Two properties keep this honest. **The registry is the allowlist.** You can only compose from skills that were published through a PR, so every agent is assembled from reviewed, owned building blocks, there's no back door that skips governance. And **composition doesn't exempt you from the framework.** Being *able* to snap five skills together doesn't mean an agent belongs there. The three steps still run, and the gates still get a veto. Composition makes agents cheap to propose, which is exactly why the gates matter more, not less, the easier it gets to assemble one, the more the discipline has to live in deciding whether to. Note the difference from escalation one more time escalation promotes a single skill

into its own runtime. Composition assembles many skills into one agent's context. Building blocks, not a promotion.


Skills Registry
Data Contract Repo Skills
SKILL.md
Data Orchestration Repo Skills
SKILL.md
dbt Africa Repo Skills
SKILL.md
Pick Subset By Description
Engineer
(coding agent)
Coordinator
Agent



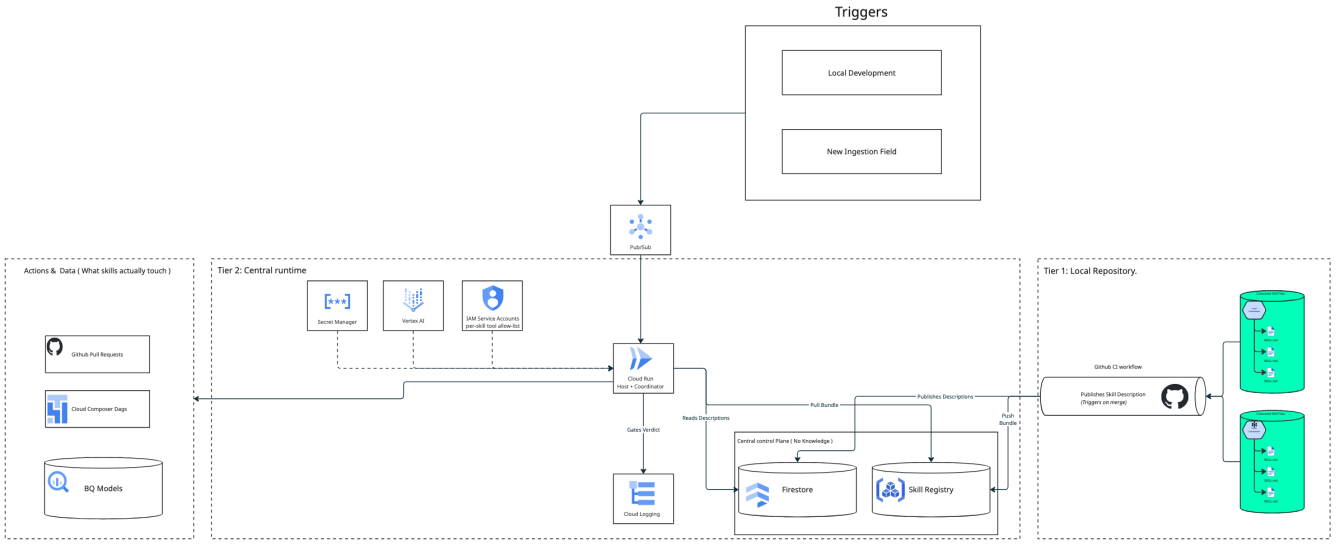
The Architecture on GCP

The two-tier design maps cleanly onto GCP services, and the mapping is what makes "distribute the knowledge, centralize only the routing" concrete rather than aspirational. Knowledge stays in the repos; one central runtime executes; everything the agent actually touches is already a GCP-managed service Credeq runs today.

Concept	GCP service	Why
Skill (knowledge), colocated	repo — GitHub	versioned and governed with the code it describes
Publish on merge	**Github Actions** trigger per repo	registration rides the existing CI, no manual step

Concept	GCP service	Why
Registry — descriptions + pointers	**Firestore**	cheap structured lookups the coordinator reads to route
Skill bundles — derived artifacts	**Artifact Registry** (or GCS)	host loads a built bundle, never clones repos at runtime
Host + coordinator runtime	**Cloud Run**	one service, scales to zero, one thing to deploy and secure
Reasoning / model	**Vertex AI** — Gemini, or Claude via Model Garden	managed inference, IAM-scoped, no key sprawl
Async triggers (Slack bug, ingestion)	**Pub/Sub** + webhook on Cloud Run	decouples the trigger from the run, gives retries
Secrets (tokens, model keys)	**Secret Manager**	no credentials baked into skills or images
Per-skill tool scoping	**IAM** service accounts / Workload Identity	the tool allowlist becomes a hard boundary, not a prompt
Open PRs	GitHub API	the agent hands back at the PR, as designed
Trigger DAGs	**Cloud Composer** (managed Airflow)	the DAG steps stay where they already live

Concept	GCP service	Why
Build / compare data	**BigQuery**	dbt targets and legacy comparisons run in-warehouse
Scoreboard — speed, gate verdicts	**Cloud Logging / Monitoring / Trace**	the "read it after it ships" number, instrumented
Escalated sub-agent	dedicated **Cloud Run** service + own IAM	independent runtime and sandbox, only when a gate demands



A local workflow (dbt dimension) never touches the central plane. The repo-local coordinator runs inside the dbt Region repo's own CI, loads its own skills, acts on BigQuery, and opens a PR. Cheap common case, no central hop.

A cross-repo workflow (Slack production bug) posts an event to **Pub/Sub**, which wakes the **Cloud Run** host. The host reads **Firestore** to match the bug against skill descriptions, loads the relevant bundles from **Artifact Registry**, reasons over them with **Vertex AI**, and dispatches to the colocated skills in handoff order, opening PRs via the GitHub API and triggering the right DAG in **Cloud Composer**. Every action runs under a per-skill **IAM** service account, so a skill can only touch what its allowlist grants.

Two GCP details do real work for the framework. **IAM per service account** turns the quality gate's "a narrow specialist can't wander into another repo" from a prompt-level hope into an enforced boundary, the skill physically cannot call what it isn't granted. And **Cloud Logging / Monitoring** is where the speed constraint stops being a claim: gate verdicts and cycle-time deltas get recorded, so "read the scoreboard after it ships" has an actual dashboard behind it. If hand-rolling the host on Cloud Run is more than you want to own, **Vertex AI Agent Engine** is the managed alternative for the runtime tier, the same split still holds, only the host's hosting changes.

How it addresses speed

Speed is the one the framework reads *after*, the scoreboard rather than a gate, so the architecture earns it in concrete ways instead of claiming it.

First, it aims the agent at the actual bottleneck, the scaffolding an engineer rebuilds for every model (Bronze and Silver configs, dedup, SCD, mapping), so the number that moves is time-to-PR for a new dimension, hands-on minutes traded for review minutes.

Second, the common case has no central hop, a local workflow goes straight to its repo's coordinator, so the frequent path pays no routing latency at all.

Third, work that used to serialize through one overloaded engineer, or one oversized agent, can run in parallel, independent repo agents and sub-agents proceeding concurrently wherever a handoff allows. And because the skill is already colocated, none of that speed is spent re-discovering the repo's scar tissue, the knowledge is in context the moment the agent starts. Whether it actually paid off isn't a matter of faith, it's the cycle-time delta on the Cloud Logging dashboard, read after it ships.

How it maintains quality

Narrow scope does most of the work here, an agent that only knows one workflow can't confidently wander into someone else's. For the rest, Google's review-and-critique pattern pairs the generator with a critic that evaluates the output against criteria and approves it, rejects it, or sends it back. That's the quality gate kept running in production.

How it reduces cost

Google's pattern guidance is blunt about the cost driving model calls. Cost scales with how many calls a pattern makes and how much context each call drags along. A hyper-specific agent keeps both small, a lean prompt, no tool bloat tax, no reasoning through five repos' worth of rules to answer a question about one. That saving lands on every single invocation, which is the kind that compounds quietly.